

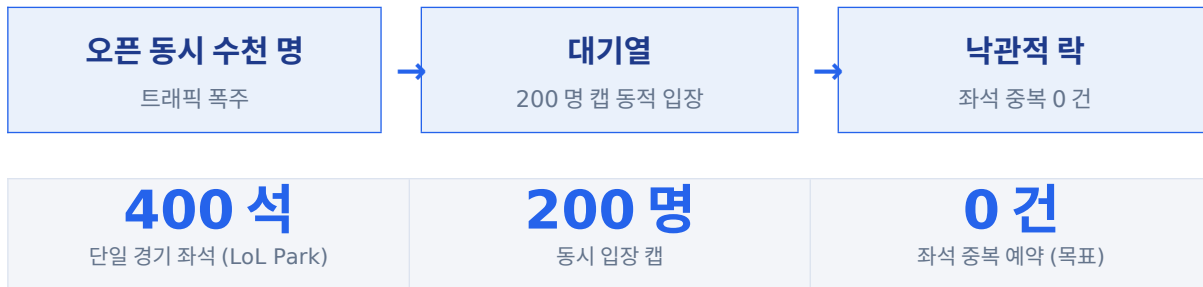
TICKETING MASTER

티케팅마스터 · 최종 프로젝트 기획안

현대이지웰 최종 프로젝트 · 1 조

대기열로 트래픽을 분산시키고, 낙관적 락으로 좌석 동시성을 보장하는 e 스포츠 티켓팅

한눈에 보기



트래픽은 대기열로 분산하고, 좌석 동시성은 그 뒤에서 낙관적 락으로 처리하는 두 단계 방어 구조

① 프로젝트 개요

항목	내용
프로젝트명	티케팅마스터 (Ticketing Master)
한 줄 소개	오픈 직후 동시 수천 명이 몰리는 트래픽을 대기열로 분산하고, DB 낙관적 락으로 좌석 중복 예약 0 건을 보장하는 e 스포츠 티켓 예매 플랫폼
도메인	e 스포츠 티켓팅 (LOL, VALORANT, OVERWATCH 등)
기준 시나리오	LCK 2026 스프링 개막전 T1 vs Gen.G, LoL Park 400 석
동시 접속	실측 최대 4,000~5,000 명 (설계 목표 1 만, 부하생성기 한계)
대기열 입장 단위	남은 좌석 수 기준 동적, 최대 200 명 캡 (10 초 주기 승격)
팀 / 기간 / 아키텍처	3 명 풀스택 / 2026.04.22 ~ 2026.06.24 / 모놀리식

핵심 목표

- 대기열로 좌석 점유 단계의 동시 트래픽을 제어된 수준으로 분산
- 좌석 동시성 제어로 중복 예약 0 건 달성
- 결제/좌석 차감 정합성 보장 (역등성 + 보상 트랜잭션)
- 부하 테스트로 낙관적 락의 한계 지점을 수치로 확인 → 데이터 기반 의사결정

② 문제 정의 — "서버는 왜 터지는가"

단순한 "수량 감소" 문제가 아닌, 실제 티켓팅 환경에서 발생하는 6 가지 장애를 카테고리로 분해한다.

카테고리	발생 상황	본 프로젝트 대응
트래픽 폭주	오픈 시각 동시 접속 폭주, 커넥션 풀 고갈	✅ 대기열 200 명 캡 동적 입장
DB 병목	동일 좌석 동시 조회/수정으로 Lock 경합	✅ 대기열 분산 + 인덱스
동시성	같은 좌석 동시 결제로 중복 예약	✅ DB 낙관적 락 (@Version)
재고 정합성	결제 도중 취소로 좌석 잠김	✅ DB 스케줄러로 점유 자동 해제
외부 연동	PG API 타임아웃, 네트워크 단절	🟡 paymentKey 기반 역등성 + 토스 연동
장애 전파	결제 모듈 장애가 전체 확산	❌ 범위 제외

💡 본 프로젝트의 무게중심

위 6 개 중 **[트래픽 폭주]**와 **[동시성]**을 핵심으로 깊게 다룬다.

대기열이 앞단에서 동시 수천 명을 200 명 이하 단위로 분산시키고, 그 뒤에서 낙관적 락이 좌석 단위 충돌을 처리하는 **"두 단계 방어"** 구조.

③ 핵심 컨셉 — 두 단계 방어

단일 기술이 아닌 "트래픽 특성에 맞는 단계별 흐름"으로 동시성에 접근한다.

단계	역할	기술
1 단계	동시 입장 인원 제한	Redis Sorted Set + 입장 토큰 (최대 200 명 캡 동적 승격)
2 단계	좌석 단위 충돌 방지	JPA @Version (낙관적 락) + 재시도

좌석 상태 흐름

AVAILABLE → RESERVED → SOLD (RESERVED → AVAILABLE 복구, SOLD → AVAILABLE 취소 복구)

- RESERVED 진입 시 점유 만료 시각(점유 후 7 분)을 기록
- RESERVED → AVAILABLE 복구 경로 두 가지
 - **본인 점유 해제:** 즉시 복구
 - **스케줄러 만료:** 30 초 주기로 만료된 좌석을 자동 복구
- 결제 완료 → SOLD 확정
- 예매 취소 → SOLD → AVAILABLE 직접 복구

④ 핵심 기능

4.1 대기열 입장 제어

오픈 직후 수천 명이 한꺼번에 몰리면 백엔드와 DB 커넥션이 바로 고갈된다. 대기열을 앞에 세워 한 번에 들어오는 인원을 줄인다.

- **핵심:** Redis 기반 대기열로 좌석 점유 동시 인원을 200 명 이하로 유지
- **입장 단위:** 10 초 주기로 남은 좌석 수에 맞춰 동적 승격(최대 200 명 캡), 매진이면 그 사이클만 쉬고 좌석이 풀리면 다음 주기에 자동 재개
- **상태 확인:** 프론트가 3 초 주기로 대기 상태를 확인해 입장 시점 감지 (Polling 방식)

4.2 좌석 점유 + 동시성

같은 좌석을 두 사람이 동시에 사면 중복 예약이 된다. 좌석마다 버전을 뒀서 먼저 잡은 사람만 성공하게 만든다.

- **핵심:** JPA 낙관적 락(@Version) — 충돌 시 짧게 1~2 회 재시도. 좌석을 잡으면 RESERVED 로 바꾸고 점유 만료 시각과 점유자를 함께 기록
- **낙관적 락 선택 이유:** 락 자원을 잡지 않아 DB 부하가 적고, 대기열로 충돌 빈도를 이미 낮춰둔 저충돌 환경에 가장 잘 맞음

방식	장점	단점 / 결정
비관적 락	안전, 단순	락 대기 누적 → 현재는 미적용
Redis 분산 락	분산 환경 대응	외부 의존성 추가 → 추후 필요 시 검토
낙관적 락 ★ (채택)	충돌 적은 환경 최적	충돌 잦으면 재시도 누적 (대기열로 빈도 제어)

✕ 의사결정 분기

부하 테스트에서 인기 좌석 충돌이 누적되면, 그 좌석만 비관적 락 또는 Redis 락으로 이원화. 그렇지 않으면 낙관적 락 그대로 유지.

4.3 결제 ↔ 좌석 정합성

결제는 됐는데 좌석 차감이 실패하면 돈만 빠진다. 결제와 확정을 단계로 나누고, 어긋나면 자동으로 되돌린다.

- **핵심:** 예약(PENDING) → 결제 성공 → 확정(CONFIRMED)으로 단계 분리, 같은 결제 요청은 한번만 처리(멥등성), 결제 후 내부 처리가 실패하면 결제를 자동 취소(보상 트랜잭션)
- **가격 보존:** 예매 시점 좌석 가격을 스냅샷으로 저장해 이후 가격이 바뀌어도 영향 없음

결제 흐름 (토스페이먼츠 위젯)

1. "결제하기" 클릭 → 토스 위젯 열림
2. 위젯에서 결제 수단 선택 (카드 / 간편결제 / 계좌이체)
3. 결제 결과를 백엔드로 전송
4. 백엔드가 토스 승인을 확인한 뒤 예매 확정

결제 수단 선택 화면은 우리 프론트가 아니라 토스 위젯이 담당

4.4 좌석 점유 만료

좌석을 잡아두고 결제하지 않으면 그 자리가 묶인다. 일정 시간이 지나면 자동으로 풀어 다시 판매한다.

- **핵심:** 점유 후 7 분이 지나면 스케줄러가 주기적으로 점유를 해제해 다시 판매 가능 상태로 복구

4.5 취소표 반환

예매를 취소하면 그 좌석이 곧바로 다시 풀려 다른 사람이 살 수 있다.

- **환불 수수료:** 경기 시작 기준 72 시간 이상 0%, 24~72 시간 10%, 24 시간 미만 취소 불가
- **처리:** 취소 시 좌석을 즉시 판매 가능 상태로 되돌리고 토스 환불 API 로 환불, 취소 사유도 함께 기록

⑤ 차별화 포인트

- **D1. 문제 정의의 구체성** — "왜 서버가 터지는가"를 6 개 카테고리로 분해, 수천 동시접속/400 석/200 명 캡 동적 배치라는 구체적 시나리오
- **D2. 기술 선택의 흐름** — 각 기술을 독립적으로 선택하지 않고, 앞 단계가 만든 환경을 전제로 다음 기술을 선택. 대기열로 동시 충돌을 낮춘 뒤, 저충돌 환경에 적합한 낙관적 락을 적용하는 단계적 설계
- **D3. 적용 → 측정 → 결정의 반복** — 처음부터 완벽한 설계를 추구하지 않고, 단계별로 측정해 다음

결정을 내림. 부하 테스트 단계에서 낙관적 락의 실제 성능을 검증하고, 인기 좌석 충돌이 누적되면 마무리 단계에서 해당 좌석만 별도 락 전략으로 분리

- **D4. 측정된 결과** — RPS/응답시간/에러율, 좌석 중복 예약 0 건, 낙관적 락 재시도 누적을 등 수치로 검증

선택 근거 요약

좌석 점유 — 낙관적 락 사용

앞단 대기열로 동시 트래픽이 200 명 수준까지 줄어든 저충돌 환경, 락 자원을 점유하지 않아 DB 부하 최소화.

⑥ 데이터 모델 요약

엔티티	역할	비고
User	회원 / 관리자	role: USER / ADMIN
Event	대회 / 리그	메인 카드 단위, 예매 정책 보유
Match	회차 (개별 경기)	예매 최소 단위, 좌석/대기열 독립
Team	참가 팀	Match 에 home/away 연결
SeatGrade / Section	등급+가격 / 구역	Event 단위 공유
Seat	개별 좌석	Match 단위, @Version + reserved_until + reserved_by
Queue	대기열 엔트리	Redis 실시간 + DB 이력
Booking / BookingSeat	예매 + 좌석 매핑	예매 이력 + 가격 스냅샷
Payment	결제	Booking 과 1:1, PG 키 보유

핵심 설계 포인트

- Event = 대회, Match = 회차 → 메인 카드는 Event, 예매는 Match
- Seat 에 @Version 컬럼 → 낙관적 락의 핵심
- BookingSeat → 예매 이력 (좌석 점유 불변량은 Seat.status + @Version 에서 강제)
- BookingSeat.seat_price → 예매 시점 가격 보존

⑦ 기술 스택 / 진행 전략

기술 스택

영역	선택
언어 / 프레임워크	Java 21, Spring Boot 3.x
DB / ORM	Oracle 23ai + Spring Data JPA + @Version
Cache / Queue	Redis 8 (Sorted Set 기반 대기열, 트래픽 분산 단계에서 본격 활용)
인증	JWT (Access: Authorization 헤더 / Refresh: Body, 쿠키 미사용)
프론트엔드	React + TypeScript, Zustand, shadcn/ui, TanStack Query
부하 테스트 / 인프라	k6 / Docker Compose / GitHub Actions

진행 단계

단계	기간	내용
1 단계 — 우선 돌아가게	Week 1~4	기본 기능 + 낙관적 락 좌석 점유 + DB 스케줄러 점유 완료, 대기열은 단순 폴링 또는 없음, Redis는 세션 정도만
2 단계 — 트래픽 분산 + 한계 측정	Week 5~7	Redis Sorted Set 대기열 + 200명 캡 동적 승격, k6 부하 테스트로 낙관적 락 한계 수치화
3 단계 — 의사결정 + 마무리	Week 8~9	결과에 따라 (a) 낙관적 락 유지 또는 (b) 인기 좌석만 이원화 발표 준비, 문서화

⑧ 검증 방법

부하 테스트 시나리오 (k6)

- A 좌석 동시성 — 인기 좌석 50 석에 1000명 동시 점유 시도
- B 대기열 스파이크 — enter 5000 RPS 스파이크 유입 흡수
- C 결제 중단 — 점유 → 예매 → 결제 중단 플로우 일관성 검증
- D 대기열 ON/OFF — 큐 ON/OFF 비교로 대기열 효과 측정
- E 좌석 조회 — 4000 VU read 부하에서 캐싱 동작 확인
- F 락 스트레스 — 1 좌석 4000명 동시 (낙관적 락 워스트케이스)

측정 지표

- RPS, 평균/p95/p99 응답시간, 에러율
- 좌석 중복 예약 발생 건수 (반드시 0)
- OptimisticLockException 발생률, 재시도 성공률
- DB Connection 사용률, Redis 메모리 사용량

🌀 프로젝트 한 줄 정의

트래픽 폭주는 대기열로 분산시키고, 좌석 동시성은 그 뒤에서 낙관적 락으로 처리하는, **두 단계 방어 구조의 e 스포츠 티켓 예매 시스템**

부록 A 구현 상세

본문에서 뺀 구현 레벨 디테일

대기열

- Redis Sorted Set(ZADD score=진입 시각) 기반, UUID 입장 토큰 발급
- 승객 스케줄러 @Scheduled(fixedDelay 10 초) — 매 주기 min(200, 남은 좌석 + 버퍼 10)명 ALLOWED 승객
- 매진 시 Redis 플래그(markSoldOut)만 ON, 좌석 복구 시 자동 해제(clearSoldOut) — 스케줄러 자체는 중단 없음
- Spring Filter 에서 Queue-Token 헤더 검증, 토큰 없이 좌석 API 접근 불가
- 최종 상태(WAITING / ALLOWED / EXPIRED)는 DB Queue 테이블에 이력 기록

좌석 조회 / 캐싱

- 2 단계 조회 — GET /matches/{id}/sections(구역 + 잔여 수) → GET .../sections/{id}/seats(좌석 + 상태)
- 잔여 카운트는 status NOT IN (SOLD, RESERVED)만 집계
- 매치 좌석 1 회 로딩 + 메모리 집계로 N+1 방지, 좌석맵 Caffeine 캐시 TTL 2 초, AWS 배포본 nginx 응답 캐시 3 초 (stale 좌석은 점유 시 @Version 이 차단)

좌석 점유 필드

- Seat.status, Seat.reserved_until(now + 7 분), Seat.reserved_by(userId), Seat.version(@Version 자동 증가)
- 충돌 시 OptimisticLockException → 1~2 회 재시도(50~100ms)

결제

- 역등 키 = 토스 paymentKey (같은 키 재호출은 기존 결과 반환)
- POST /payments/confirm 에서 토스 승인 API 호출 후 Booking(CONFIRMED) 처리
- 토스 승인 성공 후 DB 트랜잭션 실패 시 토스 cancel API 로 보상 환불, PaymentFailureRecorder 는 REQUIRES_NEW 로 분리 기록
- 가격 스냅샷은 BookingSeat.seat_price 에 보존

점유 만료 / 취소

- @Scheduled 로 reserved_until 지난 RESERVED 좌석을 30 초 주기로 AVAILABLE 복구
- Redis Keyspace Notification 은 추후 확장 옵션으로 보류 (현재는 외부 의존성 최소화)
- 취소 시 Booking.status = CANCELED, Seat SOLD → AVAILABLE 직접 복구, Booking.cancelReason 에 사유 기록

주요 인덱스

- Seat(match_id, status), Seat(match_id, section_id), Seat(match_id, seat_grade_id), Queue(match_id, status), Booking(user_id, status)

부록 B 측정 결과

부하 테스트 시나리오 A~F의 실측 결과

- A 좌석 동시성 — 50 석에 1000명 동시 → 정확히 50명 성공, 좌석 중복 0
- B 대기열 스파이크 — enter 5000 RPS 폭주를 무손실 흡수
- C 결제 종단 — 100% 완주, payment = booking = seat 수량 3종 일치
- D 대기열 ON/OFF — 큐 ON 시 좌석 점유(reserve) p95 59%↓, 대기열 효과 입증
- E 좌석 조회 — 4000 VU read를 캐싱으로 흡수
- F 락 스트레스 — 1 좌석 4000명 동시에서도 중복 0 (낙관적 락 워스트케이스 통과)